

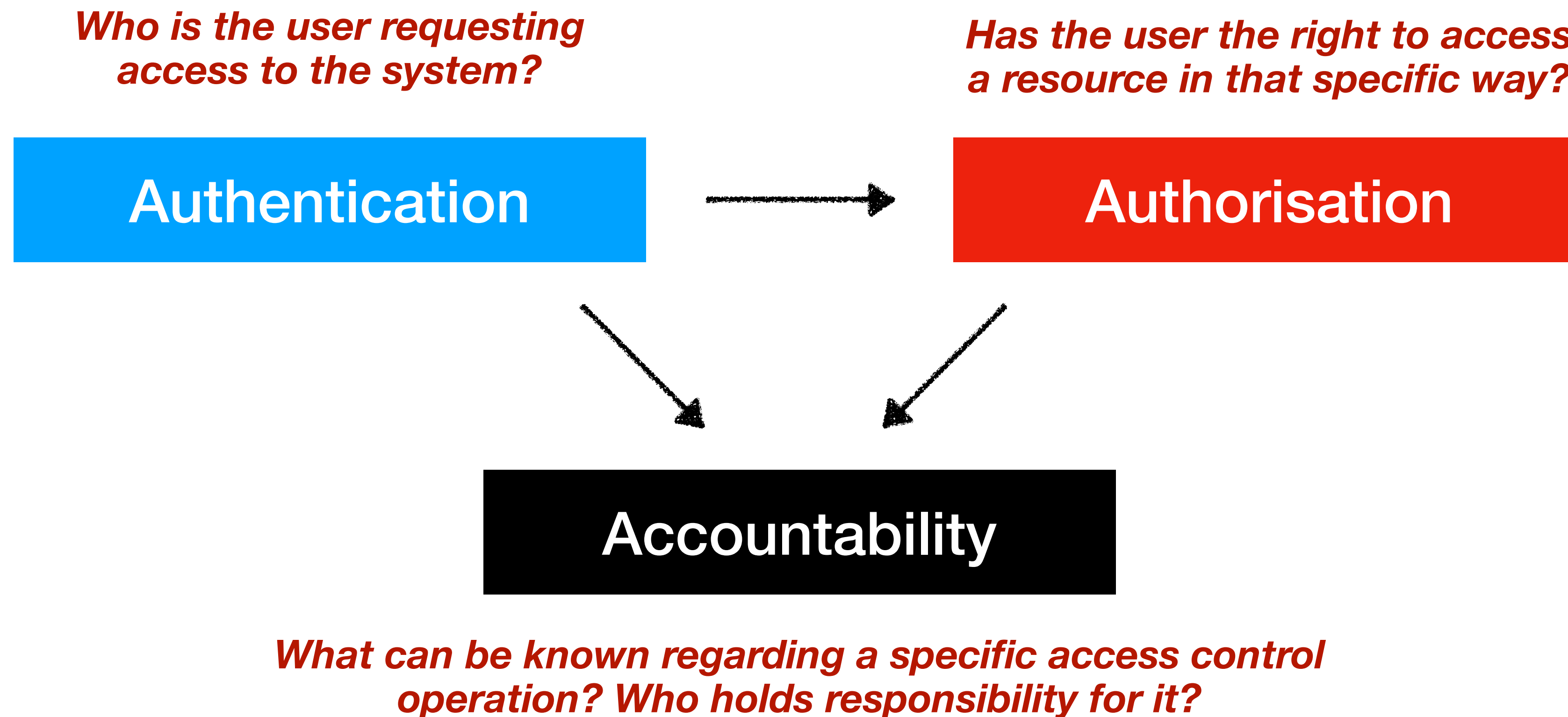
Access Control Concepts: The Linux Filesystem Case Study

Fundamentals of Access Control

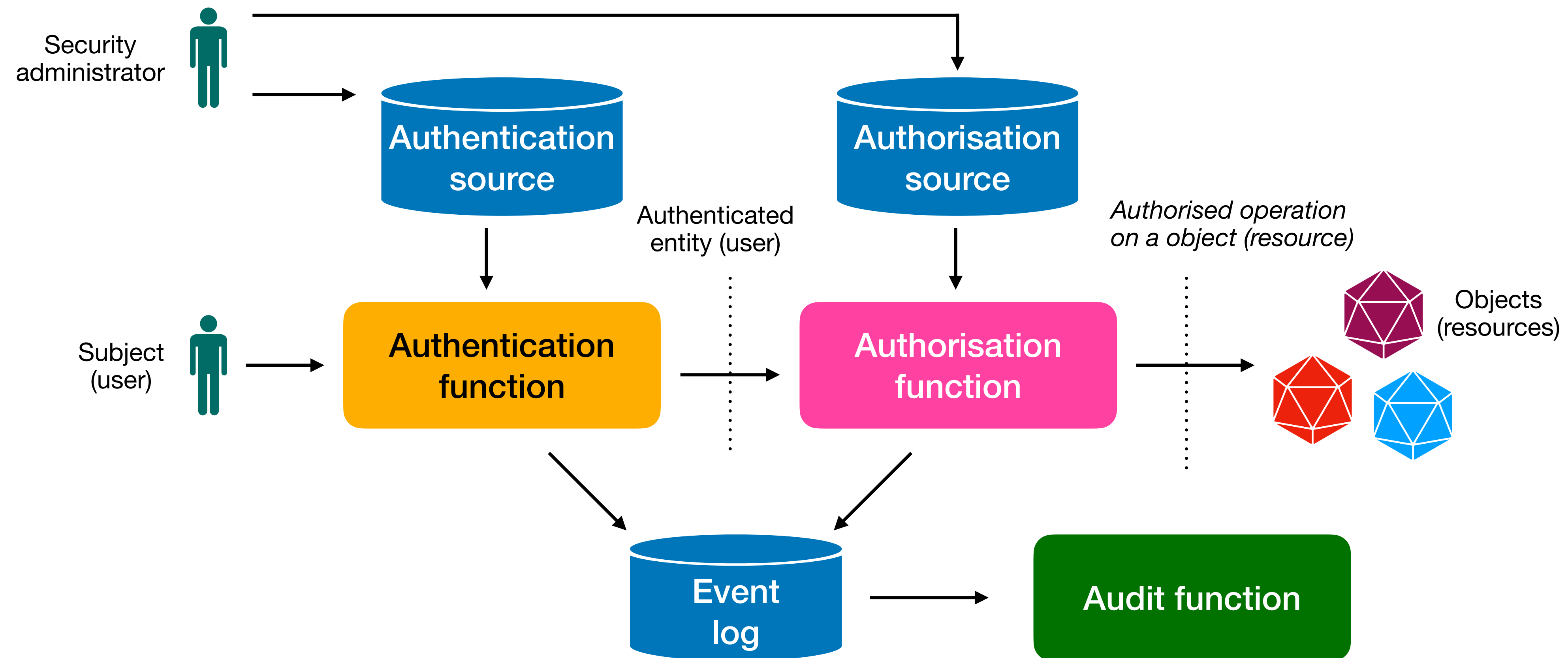
Access Control as a General Security Problem

- Shared resources require regulated use
- Access control answers: “***who may do what to which resource***”
- Fundamental to secure system design
- Applies across domains (OS, DB, distributed systems)
- Independent of implementation technology
- Enforcement must not rely on application correctness

Access Control Functions



Overview of Access Control



Access Control Functions

- **Authentication:**
 - Verification that the credentials of a user or other system entity are valid.
 - It determines who can access the system.
- **Authorisation:**
 - The granting of a right or permission to a system entity to access a system resource.
 - This function determines who is trusted for a given purpose.
- **Audit:**
 - An independent review of system to detect weaknesses, ensure compliance, and improve controls.

Access Control Functions

- **Authentication:**

- Verification that the credentials of a user or other system entity are valid.
- It determines who can access the system.

Our focus



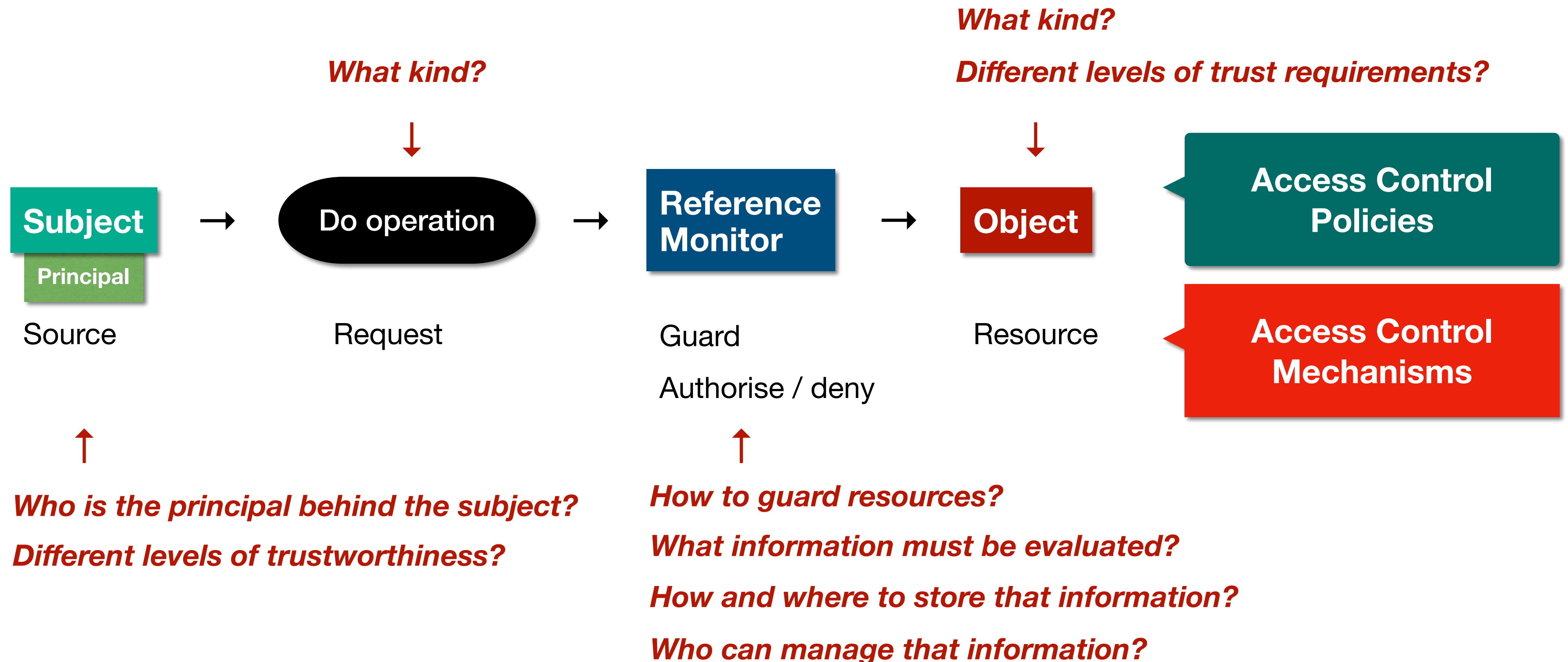
- **Authorisation:**

- The granting of a right or permission to a system entity to access a system resource.
- This function determines who is trusted for a given purpose.

- **Audit:**

- An independent review of system to detect weaknesses, ensure compliance, and improve controls.

Fundamental Authorisation Model



Core Access Control Abstractions

- **Subjects:** active principals (users, processes, services)
- **Objects:** protected resources (files, rows, APIs, memory)
- **Operations:** actions on objects (read, write, invoke, delete)
- Access decision as function of subject, object, operation
- Subjects and objects may be dynamic

Policy vs Mechanism

- **Policy:** declarative rules defining allowed actions
- **Mechanism:** infrastructure enforcing those rules
- **Policy goals:** clarity, consistency, auditability
- **Mechanism goals:** correctness, completeness, robustness
- **Policy must be enforceable by the mechanism**
- Decoupling enables reuse across systems

The Reference Monitor

Abstract Model

- Conceptual entity mediating all accesses
- Independent of system domain
- Required properties:
 - **Complete mediation**
 - **Tamper resistance**
 - **Verifiability**
- Enforces security policy decisions
- Foundation of trusted system design

The Reference Monitor

Across Domains

- Operating systems: system call mediation
- Databases: query execution checks
- Language runtimes: sandbox enforcement
- Distributed systems: API gateways
- Hardware: MMUs, IOMMUs
- Same concept, different realisations

Access Control Matrices

Conceptual to Practical Mechanisms

- **Conceptual representation** of access rights
- Rows correspond to subjects
- Columns correspond to objects
- Matrix entries list allowed operations
- **Typically sparse** in real systems
- Basis for ACLs and capability systems

Subjects

Resources			
	R1	R2	R3
Alice	Read		Print
Bob	Read	Read/Write	
Charlie	Write	Write	Print
Admin	Read/Exec	Read	Manage

Access Control Matrices

- **Conceptual representation** of access rights
- Rows correspond to subjects
- Columns correspond to objects
- Matrix entries list allowed operations
- **Typically sparse** in real systems
- Basis for ACLs and capability systems

		Resources		
		R1	R2	R3
Subjects	Alice	Read		Print
	Bob	Read	Read/Write	
	Charlie	Write	Write	Print
	Admin	Read/Exec	Read	Manage


ACL

Access Control Matrices

- **Conceptual representation** of access rights
- Rows correspond to subjects
- Columns correspond to objects
- Matrix entries list allowed operations
- **Typically sparse** in real systems
- Basis for ACLs and capability systems

Capability →

Subjects

Resources

	Resources		
	R1	R2	R3
Alice	Read		Print
Bob	Read	Read/Write	
Charlie	Write	Write	Print
Admin	Read/Exec	Read	Manage

Discretionary Access Control (DAC)

- Decisions based on subject identity
- Authority can be delegated
- Ownership-based control
- Policies change dynamically
- Mechanism Implemented via ACLs or permission bits
- Susceptible to Trojan horse attacks

Mandatory Access Control (MAC)

- Central authority defines access policy
- Subjects cannot override rules
- Subjects and objects carry security labels
- Policy enforces global invariants
- Strong confinement guarantees
- Resistant to indirect misuse

Information Flow Perspective

- Access control regulates information flow
- Read/write rules define allowed flows
- Aims at preventing confidentiality breaches and/or integrity violations
- Basis for formal security models
- Relevant beyond storage systems

Role-Based Access Control (RBAC)

- Users assigned to roles
- Roles aggregate permissions
- Sessions activate subsets of roles
- Supports separation of duties
- Simplifies administration
- Widely used in enterprises

		Roles		
Subjects		Student	Teacher	Rector
	Alice	Yes		
	Bob	Yes	Yes	
	Charlie		Yes	
	Dave		Yes	Yes

		Resources		
Roles		R1	R2	R3
	Teacher	Read	Write	Print
	Student	Read	Read/Write	
	Secretary	Write	Read	Print/Manage
	Rector	Read	Read	Manage

Role-Based Access Control (RBAC)

Constraining a Many-to-Many Relation

- **Many-to-many relations for roles and users, and for roles and resources:**
 - A user can be assigned multiple roles.
 - A role can be held by multiple users.
 - A role can grant access rights to multiple resources.
 - Access rights to a resource can be granted by multiple roles.
- A role can be treated as an object, allowing the definition of role hierarchies.
- **Possible constraints:**
 - Mutually exclusive roles (or active roles).
 - Cardinality on roles (or active roles).
 - Prerequisite role.
- Constraints can also be combined.

Attribute-Based Access Control (ABAC)

- Decisions based on attributes
- Attributes from subject, object, environment
- Policies expressed as predicates
- Highly expressive
- Enable context-aware access
- Harder to analyse and audit
- Particularly relevant in distributed systems settings
- E.g. *User authentication not authorised after working hours* (time is an environment attribute)

Capability-Based Access Control

- Authority represented as an unforgeable token
- Possession implies permission
- No global access control list
- Authority is explicit and transferable
- Revocation is non-trivial
- Contrasts with identity-based control

Capabilities in Practice (Linux and Beyond)

- Linux decomposes `root` into capabilities
 - Capabilities attached to processes
 - Checked independently of UID
 - Enforces least privilege
 - Avoids ambient authority
 - Used in microkernels and language runtimes
- Examples
 - `CAP_NET_BIND_SERVICE`
 - `getcap /usr/bin/ping`
 - File-descriptor-based access in Unix

Comparing Access Control Models

- **DAC: *Who are you?***
 - Flexible, weak confinement
 - **MAC: *What does policy allow?***
 - Strong guarantees, rigid policy
 - **RBAC: *What role are you in?***
 - Organisational scalability
 - **ABAC: *What are the current conditions?***
 - Expressiveness and context
 - **Capabilities: *What authority do you hold?***
 - Explicit, least-privilege authority via possession
- Models address **different threat models**
 - **Hybrid approaches** dominate practice

Common Conceptual Pitfalls

- Confused deputy problem
- Ambient authority
- Over-privileged principals
- Policy drift over time
- Implicit vs explicit access
- Lack of auditing

Where Operating Systems Fit

- OS as one instantiation of access control
- Kernel as reference monitor implementation
- Filesystems as object domains
- Processes as subjects
- System calls as mediation points
- Next: Linux filesystem case study

Access Control and the Security Design Principles

Access Control and Security Design Principles

- Access control is a primary mechanism for enforcing security principles
- Design principles guide how access control should be structured
- Poor access control design usually violates multiple principles at once
- Principles apply across OSs, databases, and distributed systems
- Reference monitor embodies several principles simultaneously
- Models (DAC, MAC, RBAC, ABAC) operationalise different principles

Least Privilege

- Subjects should operate with minimal required authority
- Access rights should be:
 - Narrow in scope
 - Limited in time
- Reduces impact of bugs and compromises
- Violations often appear as ambient authority
- Difficult to retrofit after system design

Least Privilege (cont.)

Access control links

- DAC over-sharing
- setuid-root programs
- Linux capabilities as privilege decomposition
- RBAC role minimisation

Complete Mediation and the Reference Monitor

- Every access must be checked
- No “fast paths” or bypasses
- Checks must occur at the enforcement boundary
- Caching must not weaken correctness
- Reference monitor formalises this principle
- Kernel mediation is one concrete instantiation

Complete Mediation and the Reference Monitor

Access control links

- System calls as mediation points
- Path traversal checks in filesystems
- LSM hooks enforcing MAC
- Database query rewriting

Economy of Mechanism and Policy Simplicity

- Simpler mechanisms are easier to verify
- Smaller trusted computing base (TCB)
- Complex policies increase misconfiguration risk
- Expressiveness vs analysability trade-off
- ABAC policies are powerful but can be opaque

Economy of Mechanism and Policy Simplicity

Access control links

- Minimal kernel checks vs user-space enforcement
- ACL complexity vs mode bits
- MAC policy size and correctness
- Overly expressive IAM rules

Separation of Privilege and Separation of Duties

- Access requires satisfying multiple independent conditions
- Reduces single points of failure
- Common in organisational security
- Can be static or dynamic
- Often enforced via roles and workflows

Separation of Privilege and Separation of Duties

Access control links

- RBAC separation-of-duties constraints
- Multi-step administrative actions
- Capabilities combined with DAC/MAC
- Distinct read vs write authorities

Fail-Safe Defaults and Deny-by-Default

- Access should be denied unless explicitly granted
- Secure failure modes matter
- Default permissions shape real security posture
- Silent failures are dangerous
- Explicit policy beats implicit assumptions

Fail-Safe Defaults and Deny-by-Default

Access control links

- Default file permissions (umask)
- LSM default-deny policies
- Capability absence implies denial
- Database default privilege sets

Linux Filesystem Access Control: Concepts in Practice

Linux Filesystem as an Access Control Domain

- Filesystem as a collection of protected objects
- **Objects:** files, directories, special files
- **Subjects:** processes (on behalf of users)
- **Operations:** read, write, execute, metadata changes
- Access decisions enforced by kernel
- Multiple access control models applied

From User Request to Kernel Decision

- User-level request via **system call**
- Transition from **user mode** to **kernel mode**
- **Path resolution (traversal)** and **i-node lookup**
- **Access control checks** performed (always enforced)
- An initial open operation returns a file descriptor if access to the resource is granted
 - A file descriptor represents an accessible filesystem resource
 - Further operations to the resource will reference that file descriptor

Subjects in Linux: Processes and Credentials

- Processes act as subjects
- Credentials attached to processes
 - Real, effective, saved UIDs
 - Primary and supplementary GIDs
- Credentials inherited across `fork/exec` (can also be changed on `exec`)
- Credentials consulted during access checks

Objects: Files, I-nodes, and Directories

- Files represented by i-nodes
- I-nodes store metadata and permissions
- Directories are special files
- Execute permission is given on directories
- Path traversal involves repeated access checks
- Object identity is distinct from the pathname

Discretionary Access Control in Linux

- Classic UNIX DAC model
- Owner, group, others
- Read, write, execute bits
- Permission checks based on identity
- Ownership implies authority
- Delegation via `chmod/chown`

Access Control Lists (ACLs)

- Extension of basic DAC
- Per-object permission entries
- Fine-grained subject specification
- Ordered evaluation of entries
- Backward-compatible with mode bits
- Increased complexity

setuid, setgid, and Privilege Transitions

- Executable files with special bits
- Effective UID/GID change on exec
- Temporary privilege elevation
- Intended to enable controlled access
- Common source of vulnerabilities
- Strong example of ambient authority

Mandatory Access Control via LSM

- Linux Security Modules framework (LSM)
- Hook-based access control
- MAC enforced alongside DAC
- Policies independent of file ownership
- Examples: SELinux, AppArmor
- Even `root` can be restricted

Order of Access Control Checks

All mediated by kernel:

1. Path traversal checks
2. DAC permission evaluation
3. Capability checks
4. LSM/MAC policy enforcement
5. Final allow or deny decision

Capabilities and Fine-Grained Privilege

- Root privileges decomposed into capabilities
- Capabilities attached to processes
- Checked independently of UID
- Reduce need for full root access
- Example: CAP_NET_BIND_SERVICE
- Principle of least privilege

Linux Filesystem as Reference Monitor

- Kernel mediates all file accesses
- No direct hardware/device access
- All security-relevant operations checked
- Combination of DAC, MAC, capabilities
- Small trusted computing base
- Concrete reference monitor instance

Common Misconfigurations and Pitfalls

- World-writable files and directories
- Incorrect directory execute bits
- Overuse of *setuid* binaries
- Misunderstood ACL semantics
- Disabled or permissive MAC policies
- Lack of auditing and logging

Summary and Conceptual Mapping

- Linux filesystem as DAC-dominated
- MAC layered via LSM
- Capabilities mitigate root omnipotence
- Kernel as reference monitor
- Models instantiated, not invented
- Concepts generalise beyond Linux

Putting It Together

- Access control is the enforcement backbone of secure design
- Models encode different principles:
 - DAC → flexibility
 - MAC → confinement
 - RBAC → administrative control
 - ABAC → contextual decisions
- Reference monitor operationalises multiple principles
- Linux combines principles pragmatically
- Design principles guide which model fits which threat

Linux Filesystem Access Control: Credentials and Privileges

Identity and Credentials in Linux

- Linux enforces access control based on process credentials
- Credentials represent who the process acts as
- Stored and managed by the kernel
- Derived from persistent credential databases
- Checked on every access decision
- Filesystem access is always process-mediated

Persistent Credential Databases (Overview)

- Credential info stored in plain-text databases `/etc/passwd`
- Historically UNIX-style design `/etc/shadow`
- Split to reduce exposure of sensitive data `/etc/group`
- Parsed by libc and authentication services `/etc/gshadow`
- Used during login and privilege changes
- Not consulted directly on every access

User database

/etc/passwd

- Maps usernames to user details
- Publicly readable, clear-text file
- Seven colon-separated fields
- UID is a unique user number (same for GID)
- A user has a primary group
- Can also have secondary groups
- UID and GID are the true identities used by the kernel (no user or group names)
- No longer stores password hashes

Fields:

1. username
2. password hash (deprecated/unused)
3. UID
4. primary GID
5. comment
6. home directory
7. login shell

User Database

/etc/passwd

```
vff — vff@ubuntu-server: ~ — ssh 127.0.0.1 -p 2222 -l vff — 80x26
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
messagebus:x:101:102:/:/nonexistent:/usr/sbin/nologin
systemd-resolve:x:992:992:systemd Resolver:/:/usr/sbin/nologin
syslog:x:103:104:/:/nonexistent:/usr/sbin/nologin
uidd:x:104:105:/:/run/uidd:/usr/sbin/nologin
tcpdump:x:105:107:/:/nonexistent:/usr/sbin/nologin
vff:x:1000:1000:vff:/home/vff:/bin/bash
sshd:x:107:65534:/:/run/sshd:/usr/sbin/nologin
systemd-coredump:x:986:986:systemd Core Dumper:/:/usr/sbin/nologin
~
```

24,1 All

Shadow User Database

/etc/shadow

- Stores password hashes and ageing data
- Readable only by `root`
- Decouples authentication secrets from identity
- Supports password expiration and lockout
- Essential for offline attack resistance
- Access controlled via filesystem permissions

Shadow User Database and Password Ageing

/etc/shadow

- **Last Password Change:** The number of days since the Unix epoch (Jan 1, 1970) when the password was last modified.
- **Minimum Password Age:** The minimum number of days a user must wait before they can change their password again.
- **Maximum Password Age:** The maximum number of days a password remains valid before the user is forced to change it.
- **Warning Period:** The number of days before expiration during which the user receives a warning message at login.
- **Inactivity Period:** The number of days after a password expires before the account is automatically disabled.
- **Account Expiration Date:** An absolute date (in days since the epoch) when the account itself expires and becomes unusable, regardless of the password's state.

Shadow User Database

/etc/shadow



```
vff — vff@ubuntu-server: ~ — ssh 127.0.0.1 -p 2222 -l vff — 80x26
root:*:20306:0:99999:7:::
daemon:*:20306:0:99999:7:::
bin:*:20306:0:99999:7:::
sys:*:20306:0:99999:7:::
sync:*:20306:0:99999:7:::
games:*:20306:0:99999:7:::
man:*:20306:0:99999:7:::
lp:*:20306:0:99999:7:::
mail:*:20306:0:99999:7:::
news:*:20306:0:99999:7:::
uucp:*:20306:0:99999:7:::
backup:*:20306:0:99999:7:::
nobody:*:20306:0:99999:7:::
systemd-network:!:20306:0:0:0:
systemd-timesync:!:20306:0:0:0:
dhcpcd:!:20306:0:0:0:
messagebus:!:20306:0:0:0:
systemd-resolve:!:20306:0:0:0:
syslog:!:20306:0:0:0:
uuidd:!:20306:0:0:0:
tcpdump:!:20306:0:0:0:
vff:$y$j9T$3Y4vpwdGIzPpvvhXkV6d80$3S36vs4c4fgHzPH06udf03tY9Z0cYINA/cBDStIwT19:20
494:0:99999:7:::
sshd:!:20381:0:0:0:
systemd-coredump:!:20382:0:0:0:
```

24,1 All

Groups and Supplementary Groups

- Groups represent collections of users
- Primary group from `/etc/passwd`
- Supplementary groups from `/etc/group`
- Kernel stores group list in credentials
- Used in DAC permission checks
- Maximum group count enforced by kernel

Group Database

/etc/group and /etc/gshadow

- /etc/group:
 - Group name, GID, member list
- /etc/gshadow:
 - Group passwords (rarely used)
 - Administrative group control
- Similar split as `passwd/shadow`
- Group changes require privileges
- Misconfiguration can leak access

Group Database

/etc/group

```
vff — vff@ubuntu-server: ~ — ssh 127.0.0.1 -p 2222 -l vff — 80x26
root:*:20306:0:99999:7:::
daemon:*:20306:0:99999:7:::
bin:*:20306:0:99999:7:::
sys:*:20306:0:99999:7:::
sync:*:20306:0:99999:7:::
games:*:20306:0:99999:7:::
man:*:20306:0:99999:7:::
lp:*:20306:0:99999:7:::
mail:*:20306:0:99999:7:::
news:*:20306:0:99999:7:::
uucp:*:20306:0:99999:7:::
backup:*:20306:0:99999:7:::
nobody:*:20306:0:99999:7:::
systemd-network:!:20306:0:0:0:
systemd-timesync:!:20306:0:0:0:
dhcpcd:!:20306:0:0:0:
messagebus:!:20306:0:0:0:
systemd-resolve:!:20306:0:0:0:
syslog:!:20306:0:0:0:
uuid:!:20306:0:0:0:
tcpdump:!:20306:0:0:0:
vff:$y$j9T$3Y4vpwdGIzPpvvhXkV6d80$3S36vs4c4fgHzPH06udf03tY9Z0cYINA/cBDStIwT19:20
494:0:99999:7:::
sshd:!:20381:0:0:0:
systemd-coredump:!:20382:0:0:0:
```

24,1 All

From Login to Process Credentials

- User authenticates (PAM)
- Credentials loaded from databases
- Kernel assigns:
 - real UID
 - effective UID
 - saved UID
 - GIDs
- Credentials inherited on fork
- Possibly modified on exec

Real, Effective, and Saved UIDs

- **Real UID:**
 - Identifies the user
- **Effective UID:**
 - Used for access control checks
- **Saved UID:**
 - Allows temporary privilege dropping
- Central to privilege transitions
- Enables controlled privilege elevation
- **Same applies to groups: RGID, EGID and SGID**

Saved UID

Example

- **Startup:** A user (UID 1000) runs a *setuid-root* program.
 - `ruid=1000, euid=0, suid=0`
- **Dropping Privileges:** The program calls `seteuid(1000)` to perform unprivileged tasks safely.
 - `ruid=1000, euid=1000, suid=0`
- **Regaining Privileges:** The program calls `seteuid(0)` to finish its work. This is allowed only because `suid` is still 0.

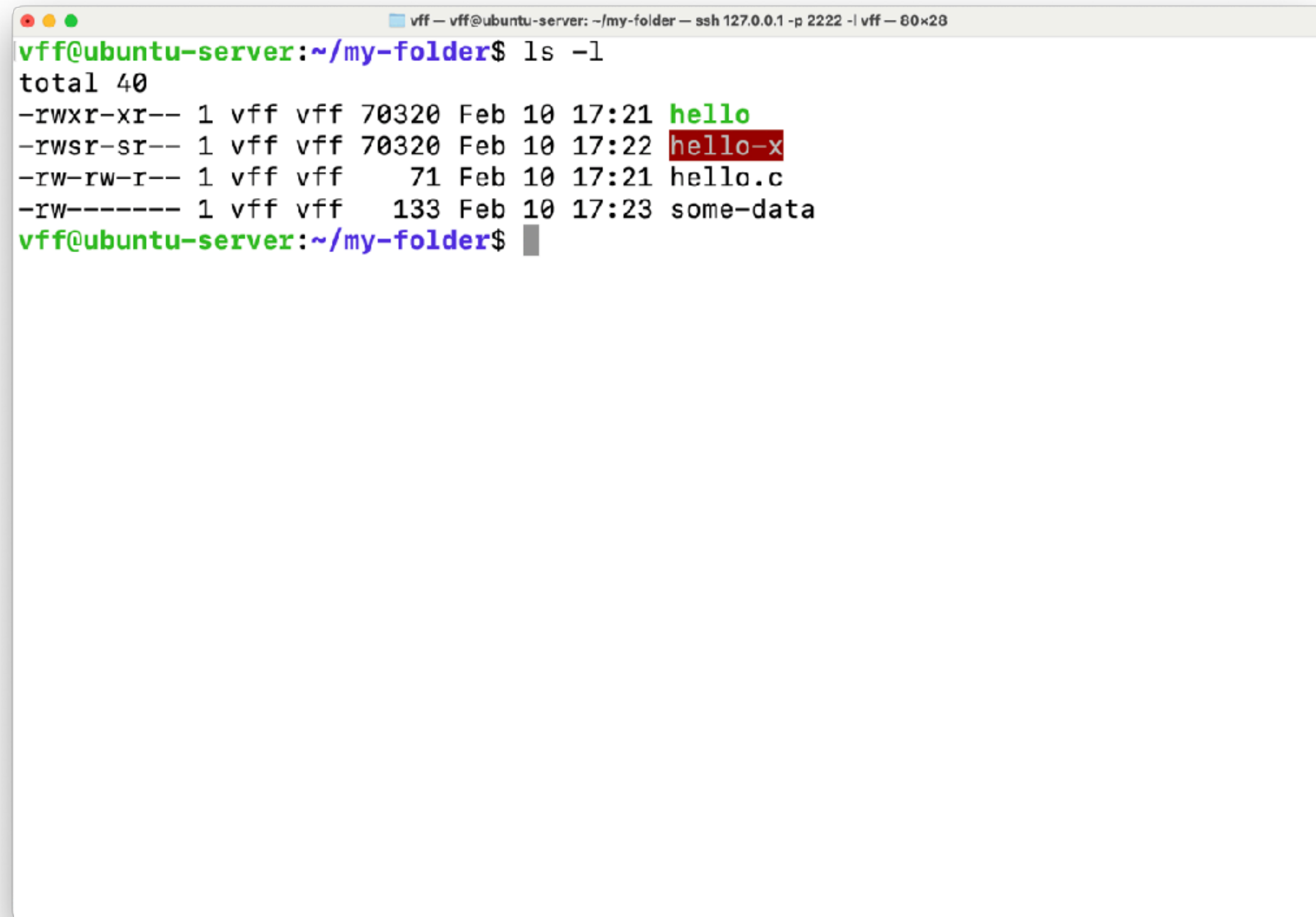
Filesystem Permission Set (DAC)

- Classic UNIX permission model
- Three subject classes:
 - user (user owning the resource)
 - group (of users sharing the resource)
 - others (all other users)
- Three operations:
 - read
 - write
 - execute

Filesystem Permission Set (DAC)

- Evaluated in strict order:
 - If user is the *owner* of the resource, consider the respective permission set.
 - Else, if user belongs to the *group* of the resource, consider the respective permission set.
 - Otherwise, consider the permissions set for *others*.
- Checked using effective UID/GID

Filesystem Permission Set (DAC)



```
vff@ubuntu-server: ~/my-folder — ssh 127.0.0.1 -p 2222 -l vff — 80x20
vff@ubuntu-server:~/my-folder$ ls -l
total 40
-rwxr-xr-- 1 vff vff 70320 Feb 10 17:21 hello
-rwsr-sr-- 1 vff vff 70320 Feb 10 17:22 hello-x
-rw-rw-r-- 1 vff vff 71 Feb 10 17:21 hello.c
-rw----- 1 vff vff 133 Feb 10 17:23 some-data
vff@ubuntu-server:~/my-folder$
```


Directory Permissions Semantics

- **Read:**
 - list directory entries
- **Write:**
 - create/remove entries
- **Execute:**
 - traverse directory
- Execute required for any access inside
- Common source of misconfiguration
- Non-intuitive but security-critical

On Path Resolution and I-node Lookup

- When a process calls something like:
 - `open("/home/alice/docs/report.txt", O_RDONLY);`
- The kernel performs path resolution (aka *traversal*) as shown in the next slide

On Path Resolution and I-node Lookup

1. Choose a starting point

- / for absolute paths
- current working directory for relative paths

2. Split the path into components

- `home → alice → docs → report.txt`

3. Iteratively traverse directories

- For each directory component:
 - Look up the name in the directory
 - Check execute (x) permission on that directory

- Deny access immediately if traversal is not allowed

4. Resolve the final component

- Locate the i-node for `report.txt`
- Apply the requested operation (read, write, execute)

5. Apply final access checks

- DAC (permissions / ACLs)
- Capabilities
- LSM / MAC hooks

Access Control Lists (ACLs)

- Extend basic permission bits
- Allow per-user and per-group entries
- Evaluated before “others”
- Default ACLs for directories
- Backward-compatible with mode bits
- Increased expressive power, higher complexity

Passwords and the `passwd` Program

- `passwd` modifies `/etc/shadow`
- Requires root privileges
- Implemented as *setuid-root*
- Performs:
 - authentication
 - policy checks
 - atomic updates
- Illustrates controlled privilege elevation

The `setuid` and `setgid` Mechanism

- Special permission bits on executables
- Change effective UID/GID on execution
- Enables access to protected resources
- Introduces ambient authority
- Historically common, now discouraged
- Frequent source of vulnerabilities

sudo vs setuid

- `sudo` provides controlled privilege escalation
- Policy-based (not file-based)
- Authentication before elevation
- Fine-grained command restrictions
- Logging and auditing support
- Reduces need for `setuid` binaries

sudoers Policy Model

- Rules specify:
 - who
 - may run what
 - as which user
- Supports host-based restrictions
- Supports command whitelisting
- Timeout-based credential caching
- RBAC-like structure

Privilege, Credentials, and Access Control Models

- DAC enforced via UID/GID
- Groups enable controlled delegation
- `setuid` bypasses DAC intentionally
- `sudo` externalises policy
- Capabilities refine root privileges
- MAC (LSM) adds orthogonal enforcement

Common Misconfigurations and Risks

- World-writable directories
- Excessive group membership
- Over-broad sudo rules
- Forgotten *setuid* binaries
- ACL misunderstandings
- Weak password policies

Summary

- Linux credentials form the basis of DAC
- Identity stored persistently, enforced dynamically
- Permission bits are simple but subtle
- *setuid* is powerful and dangerous
- `sudo` improves policy control
- All mechanisms instantiate abstract access control concepts